Cyberscope

A TAC Security Company

Audit Report

Viruex

October 2026

Files EVM-PYGT.sol, TRON-PYGT.sol, router-split-paymaster.sol, SplitPayment.sol

Audited by © cyberscope

Table of Contents

Table of Contents **1**
Risk Classification **3**
Review **4**
Audit Updates 4
Source Files 4
Overview **5**
PYGT Tokens 5
Split Payments 5
Account-Abstraction Sponsorship 5
Paymaster Administration 6
Findings Breakdown **7**
Diagnostics **8**
BC - Blacklists Addresses 9
Description 9
Recommendation 9
Team Update 10
BT - Burns Tokens 11
Description 11
Recommendation 11
Team Update 12
MT - Mints Tokens 13
Description 13
Recommendation 13
Team Update 14
RSRA - Renounced Signer Remains Authorized 15
Description 15
Recommendation 16
Team Update 16
ST - Stops Transactions 17
Description 17
Recommendation 18
Team Update 18
ZSAIS - Zero Signer Accepts Invalid Signatures 19
Description 19
Recommendation 19
CMBC - Constructor Mint Bypasses Cap 20
Description 20
Recommendation 20
CCR - Contract Centralization Risk 21

Description 21
Recommendation 22
Team Update 22
FBCBS - Freeze Blocks Compliance Burn Seizure 23
Description 23
Recommendation 23
HSSA - High S Signatures Accepted 24
Description 24
Recommendation 24
NAAD - Nominal Amounts Assumed Delivered 25
Description 25
Recommendation 25
Team Update 26
PBAP - Permit Bypasses Account Permissions 27
Description 27
Recommendation 27
SSTW - Silent Success Token Withdrawals 28
Description 28
Recommendation 28
Team Update 29
SIDC - Sponsorship Ignores Delegate Changes 30
Description 30
Recommendation 30
Team Update 30
Functions Analysis **31**
Summary **34**
Disclaimer **35**
About Cyberscope **36**

Risk Classification

The criticality of findings in Cyberscope's smart contract audits is determined by evaluating multiple variables. The two primary variables are:

1. **Likelihood of Exploitation**: This considers how easily an attack can be executed, including the economic feasibility for an attacker.
2. **Impact of Exploitation**: This assesses the potential consequences of an attack, particularly in terms of the loss of funds or disruption to the contract's functionality.

Based on these variables, findings are categorized into the following severity levels:

1. **Critical**: Indicates a vulnerability that is both highly likely to be exploited and can result in significant fund loss or severe disruption. Immediate action is required to address these issues.
2. **Medium**: Refers to vulnerabilities that are either less likely to be exploited or would have a moderate impact if exploited. These issues should be addressed in due course to ensure overall contract security.
3. **Minor**: Involves vulnerabilities that are unlikely to be exploited and would have a minor impact. These findings should still be considered for resolution to maintain best practices in security.
4. **Informative**: Points out potential improvements or informational notes that do not pose an immediate risk. Addressing these can enhance the overall quality and robustness of the contract.

Severity	Likelihood / Impact of Exploitation
Critical	Highly Likely / High Impact
Medium	Less Likely / High Impact or Highly Likely/ Lower Impact
Minor / Informative	Unlikely / Low to no Impact

Review

Audit Updates

Initial Audit	10 Sep 2026 https://github.com/cyberscope-io/audits/blob/main/pygt/v1/audit.pdf
Corrected Phase 2	01 Oct 2026

Source Files

Filename	SHA256
src/router-paymaster.sol	f766bf08479d5b40fa8fa0b21e05a7f08c173dea5a8c7956650da3a787419e0f
src/TRON-PYGT.sol	f842ce86d49b797a0673cf2e5fb3ccbb7c9e1f7833c218be2dd58857aaf72c01
src/SplitPayment.sol	dfe0a8bfd5b45160ab687bce02daa750f28c604a461f0109eb44b6e39ce6adb5
src/EVM-PYGT.sol	e5d464dc349be923a76cf80ced923d86e002aed600d8b413123f7b0f6dd3e0ed

Overview

The Viruex contracts implement PYGT tokens for EVM and Tron networks, a payment-splitting router, and an ERC-4337 paymaster that sponsors approved account-abstraction operations.

PYGT Tokens

The EVM and Tron PYGT contracts provide equivalent ERC-20 and TRC-20 token functionality. They support permit-based approvals, owner-controlled minting, per-address lifetime minting limits, holder-initiated burns, and recovery of unrelated tokens accidentally sent to the contracts.

Both implementations include centralized compliance controls. The owner can pause token activity, freeze accounts, blacklist addresses, and burn tokens held by blacklisted accounts. Frozen accounts cannot transfer, receive, or burn tokens, while blacklisted accounts cannot transfer, receive, or receive newly minted tokens.

Split Payments

The `SplitPayment` contract divides a token payment between a primary recipient and an optional fee wallet. The caller supplies the token, recipient, payment amount, fee wallet, and fee amount.

Tokens are pulled directly from the caller using `transferFrom`. The contract supports standard tokens that return `true` and non-standard tokens that return no data. If either transfer reports failure, the complete transaction reverts.

Account-Abstraction Sponsorship

The `SimplePaymasterV8` contract sponsors ERC-4337 UserOperations approved by a designated off-chain signer. The signed approval binds the operation's sender, nonce, calldata, gas parameters, chain, paymaster, and validity interval.

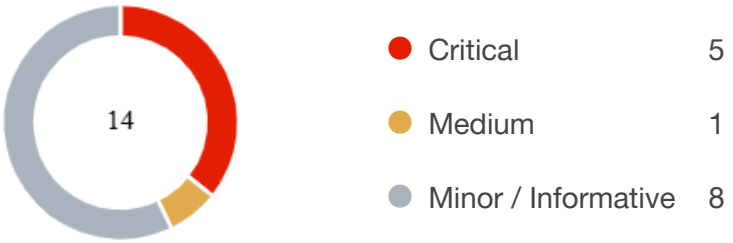
Only the configured EntryPoint can call validation and post-operation functions. Successful validation returns an empty context, so the current sponsorship model requires no post-operation settlement or user reimbursement.

Paymaster Administration

Administrators manage the paymaster's EntryPoint deposit and stake, including depositing funds, withdrawing deposits, adding or unlocking stake, and withdrawing unlocked stake. They can also withdraw native currency held directly by the contract.

Token recovery may be performed by either an administrator or an account holding `WITHDRAWER_ROLE`. Signing authority is managed independently through `SIGNER_ROLE`, while the active signer address determines which off-chain approvals the paymaster accepts.

Findings Breakdown



Severity	Unresolved	Acknowledged	Resolved	Other
Critical	0	5	0	0
Medium	1	0	0	0
Minor / Informative	4	4	0	0

Diagnostics

Critical Medium Minor / Informative

Severity	Code	Description	Status
Critical	BC	Blacklists Addresses	Acknowledged
Critical	BT	Burns Tokens	Acknowledged
Critical	MT	Mints Tokens	Acknowledged
Critical	RSRA	Renounced Signer Remains Authorized	Acknowledged
Critical	ST	Stops Transactions	Acknowledged
Medium	ZSAIS	Zero Signer Accepts Invalid Signatures	Unresolved
Minor / Informative	CMBC	Constructor Mint Bypasses Cap	Unresolved
Minor / Informative	CCR	Contract Centralization Risk	Acknowledged
Minor / Informative	FBCBS	Freeze Blocks Compliance Burn Seizure	Unresolved
Minor / Informative	HSSA	High S Signatures Accepted	Unresolved
Minor / Informative	NAAD	Nominal Amounts Assumed Delivered	Acknowledged
Minor / Informative	PBAP	Permit Bypasses Account Permissions	Unresolved
Minor / Informative	SSTW	Silent Success Token Withdrawals	Acknowledged
Minor / Informative	SIDC	Sponsorship Ignores Delegate Changes	Acknowledged

BC - Blacklists Addresses

Criticality	Critical
Location	evm-PYGT.sol#L125,141,202 tron-PYGT.sol#L126,142,203
Status	Acknowledged

Description

The contract owner has the authority to stop addresses from transactions. The owner may take advantage of it by calling the `blacklistAddress` and `freeze` function.

```
function freeze(address account) external onlyOwner {
    frozen[account] = true;
    emit Frozen({account: account});
}
...
function blacklistAddress(address account) external onlyOwner {
    blacklist[account] = true;
    emit Blacklisted({account: account});
}
...
function complianceBurn(address from, uint256 amount) external
onlyOwner {
    if (!blacklist[from]) {
        revert AccountNotBlacklisted(from);
    }
    _burn(from, amount);
    emit Burn({from: from, amount: amount});
}
```

Recommendation

The team should carefully manage the private keys of the owner's account. We strongly recommend a powerful security mechanism that will prevent a single user from accessing the contract admin functions.

Temporary Solutions:

These measurements do not decrease the severity of the finding

- Introduce a time-locker mechanism with a reasonable delay.

- Introduce a multi-signature wallet so that many addresses will confirm the action.
- Introduce a governance model where users will vote about the actions.

Permanent Solution:

- Renouncing the ownership, which will eliminate the threats but it is non-reversible.

Team Update

The team has acknowledged that this is not a security issue and states:

This behavior is intentional — the token requires these functions to comply with regulatory and governmental requirements. To mitigate centralization risks, the owner address is a Safe multisig wallet, not an EOA.

We cannot renounce ownership because the ability to act (e.g., freeze or blacklist) is legally required for compliance with court orders and government determinations. Additionally, we already use Ownable2Step to mitigate the risk of accidentally transferring ownership to an incorrect address.

BT - Burns Tokens

Criticality	Critical
Location	evm-PYGT.sol#L141,202 tron-PYGT.sol#L142,203
Status	Acknowledged

Description

The contract owner has the authority to burn tokens from a specific address. The owner may take advantage of it by calling `blacklistAddress` on any holder and then `complianceBurn` on that same address. As a result, the targeted address will lose the corresponding tokens.

The blacklist requirement provides no meaningful restriction, since the owner controls that mapping through an unguarded setter and can set the flag in the same transaction that performs the burn. There is no cap on the amount, no delay between flagging and destruction, and no mechanism for the holder to contest or reverse the action, so any balance may be removed at the owner's discretion.

```
function complianceBurn(address from, uint256 amount) external
onlyOwner {
    if (!blacklist[from]) {
        revert AccountNotBlacklisted(from);
    }
    _burn(from, amount);
    emit Burn({from: from, amount: amount});
}
```

Recommendation

The team should carefully manage the private keys of the owner’s account. We strongly recommend a powerful security mechanism that will prevent a single user from accessing the contract admin functions.

Temporary Solutions:

These measurements do not decrease the severity of the finding

- Introduce a time-locker mechanism with a reasonable delay.

- Introduce a multi-signature wallet so that many addresses will confirm the action.
- Introduce a governance model where users will vote about the actions.

Permanent Solution:

- Renouncing the ownership, which will eliminate the threats but it is non-reversible.

Team Update

The team has acknowledged that this is not a security issue and states:

The same reasoning from BC applies here — this function is intentionally included to comply with governmental orders (e.g., freezing and seizing funds linked to illicit activity). The owner address is a Safe multisig wallet, providing strong mitigation against unilateral abuse. We use Ownable2Step to prevent ownership transfer mistakes, and we cannot renounce ownership because it is essential for regulatory obligations.

MT - Mints Tokens

Criticality	Critical
Location	evm-PYGT.sol#L159,170 tron-PYGT.sol#L160,171
Status	Acknowledged

Description

The owner can mint tokens to arbitrary addresses and can increase `mintCapPerAddress` without a protocol-wide supply limit. The per-address cap does not restrict aggregate issuance because tokens may be minted across multiple accounts, and the owner can raise the cap at any time. A compromised or malicious owner could therefore substantially inflate the supply and dilute existing holders.

```
function setMintCap(uint256 newCap) external onlyOwner {
    mintCapPerAddress = newCap;
}

function mint(address to, uint256 amount) external onlyOwner
whenNotPaused {
    if (totalMintedBy[to] + amount > mintCapPerAddress) revert
MintCapExceeded(...);
    totalMintedBy[to] += amount;
    _mint(to, amount);
}
```

Recommendation

The team should carefully manage the private keys of the owner's account. We strongly recommend a powerful security mechanism that will prevent a single user from accessing the contract admin functions.

Temporary Solutions:

These measurements do not decrease the severity of the finding

- Introduce a time-locker mechanism with a reasonable delay.
- Introduce a multi-signature wallet so that many addresses will confirm the action.
- Introduce a governance model where users will vote about the actions.

Permanent Solution:

- Renouncing the ownership, which will eliminate the threats but it is non-reversible.

Team Update

The team has acknowledged that this is not a security issue and states:

Upon review, this per-address cap constant doesn't serve a meaningful purpose in practice. It was inherited from the PYGT legacy contract. We are evaluating whether to remove it, following the model of USDT on Ethereum, which has no such cap.

RSRA - Renounced Signer Remains Authorized

Criticality	Critical
Location	router-paymaster.sol#L84,92,169
Status	Acknowledged

Description

The inherited `renounceRole` removes `SIGNER_ROLE` through `_revokeRole`, bypassing the paymaster's `revokeRole` override. The separate `signer` address remains unchanged, and validation accepts fresh signatures from an account that no longer holds the role.

Additionally, revoking any account clears the active signer. The override sets `signer` to zero whenever the requested role is `SIGNER_ROLE`, regardless of whether the account is the active signer or held that role at all.

Furthermore granting a replacement leaves stale memberships. Role membership and events therefore cease to reliably describe who can authorize sponsorship.

```
function grantRole(
    bytes32 role,
    address account
) public override(AccessControl) {
    super.grantRole(role, account);
    if (role == SIGNER_ROLE) signer = account;
}

function revokeRole(
    bytes32 role,
    address account
) public override(AccessControl) {
    super.revokeRole(role, account);
    if (role == SIGNER_ROLE) signer = address(0);
}
...
```

```
if (ecrecover(hash, v, r, s) != signer) {
    return ("", _packValidationData(true, validUntil, validAfter));
}
```

Recommendation

Keep effective signing authority synchronized with role membership across every role-removal path, and consider checking current role membership during validation.

Team Update

The team has acknowledged that this is not a security issue and states:

Indeed, there was an unused state variable that previously stored the addresses holding the `SIGNER_ROLE`. We opted to replace it with a single `signer` variable because it consumes approximately half the gas compared to calling the `hasRole` function on each validation.

ST - Stops Transactions

Criticality	Critical
Location	evm-PYGT.sol#L125,141,246 tron-PYGT.sol#L126,142,246
Status	Acknowledged

Description

The contract owner has the authority to block any account from transferring, receiving, or burning tokens, without notice or restriction. The owner may take advantage of it by calling `freeze` or `blacklistAddress` on arbitrary holders, including every address except their own. Since both flags are enforced inside the `_update` hook that every balance change passes through, an affected holder retains their balance but loses all ability to move it, and only the owner can reverse the state.

Nothing prevents the target from being a liquidity pool. Freezing or blacklisting the Uniswap pair address causes every swap to revert in both directions, since a sell sets the pair as `to` and a buy sets it as `from`, leaving the pooled liquidity stranded and the market with no exit. The freeze flag is the stricter of the two and blocks the burn path as well, so a frozen holder cannot even destroy their own tokens. Combined with `pause`, which halts all transfers contract-wide, the owner can render the token permanently non-transferable for all users. As a result, the contract may operate as a honeypot.

```
if (frozen[from] || frozen[to]) {
    revert AccountFrozen(frozen[from] ? from : to);
}
if (blacklist[from] && to != address(0)) {
    revert AccountBlacklisted(from);
}
if (blacklist[to]) {
    revert AccountBlacklisted(to);
}
```

Recommendation

The team should carefully manage the private keys of the owner's account. We strongly recommend a powerful security mechanism that will prevent a single user from accessing the contract admin functions.

Temporary Solutions:

These measurements do not decrease the severity of the finding

- Introduce a time-locker mechanism with a reasonable delay.
- Introduce a multi-signature wallet so that many addresses will confirm the action.
- Introduce a governance model where users will vote about the actions.

Permanent Solution:

- Renouncing the ownership, which will eliminate the threats but it is non-reversible.

Team Update

The team has acknowledged that this is not a security issue and states:

But this is a feature, not a bug.

- *All control mechanisms are *documented in the natspec* of the contract.**
- *The functions are* `public` / `external` *with *transparent on-chain events* (* `Frozen` , `Blacklisted` , `Paused` , `Unfrozen` , `Unblacklisted`).**
- ***There is no obfuscated or hidden code.***

As documented in the contract, `initialOwner` *must not be an EOA (single key)*. It must be a **Gnosis Safe multisig**, following Tether's standard.*

Centralization is necessary to:

- *Comply with court orders (funds blocking).*
- *Meet regulatory sanctions (OFAC, etc.).*
- *Prevent fraud and theft (emergency freezing).*
- *Operate within the law in jurisdictions requiring KYC/AML.*

ZSAIS - Zero Signer Accepts Invalid Signatures

Criticality	Medium
Location	router-paymaster.sol#L97,169
Status	Unresolved

Description

The paymaster recovers the signing address with the native ecrecover precompile, which returns the zero address for an invalid signature instead of reverting. The result is compared directly against signer, and nothing prevents signer from being zero. The revokeRole override sets signer to the zero address whenever any account is revoked from SIGNER_ROLE. A routine rotation that grants a new signer and then revokes the old one therefore leaves the contract in that state.

While signer is zero, any 65-byte signature that fails recovery passes validation. Any user can then attach paymasterAndData with chosen gas values and have UserOperations sponsored from the paymaster's EntryPoint deposit. A user who submits the operation as their own bundler receives the gas fees paid from that deposit, so the deposit can be drained to the attacker.

```
function revokeRole(bytes32 role, address account) public
override(AccessControl) {
    super.revokeRole(role, account);
    if (role == SIGNER_ROLE) signer = address(0);
}
...
if (ecrecover(hash, v, r, s) != signer) {
    return ("", _packValidationData(true, validUntil, validAfter));
}

return ("", _packValidationData(false, validUntil, validAfter));
```

Recommendation

The team is advised to ensure that a zero recovery result is never treated as a valid approval, and that no role change can leave the active signer as the zero address. This will prevent sponsorship from opening to arbitrary users during signer rotation.

CMBC - Constructor Mint Bypasses Cap

Criticality	Minor / Informative
Location	evm-PYGT.sol#L104,174 tron-PYGT.sol#L105,175
Status	Unresolved

Description

The constructor mints the initial supply to `initialOwner` with a direct `_mint` call. The amount is not checked against `mintCapPerAddress` and is not recorded in `totalMintedBy`. The initial supply can therefore exceed the per address cap, and the owner address can later receive a further full cap through `mint()`. As a result, `totalMintedBy` does not reflect the tokens actually issued to that address.

```
constructor(
    address initialOwner,
    uint256 initialSupplyWhole,
    uint256 capPerAddress
)
    ERC20("PYGT", "PYGT")
    ERC20Permit("PYGT")
    Ownable(initialOwner)
{
    _mint(initialOwner, initialSupplyWhole * 1e18);
    mintCapPerAddress = capPerAddress;
}
```

Recommendation

The team is advised to ensure the initial supply is accounted for consistently with the per-address minting limit, or to document that it is excluded. This will keep totalMintedBy an accurate record of issuance.

CCR - Contract Centralization Risk

Criticality	Minor / Informative
Location	evm-PYGT.sol#L111,141,159,170,202,221 tron-PYGT.sol#L112,142,160,171,203,221
Status	Acknowledged

Description

Both token contracts concentrate significant authority in the owner. The owner can pause token activity, blacklist holders, change minting limits, issue new tokens, burn balances belonging to blacklisted accounts, and recover external tokens. Although two-step ownership transfer reduces accidental transfers, it does not limit how the active owner exercises these powers.

A compromised or malicious owner could disrupt transfers, censor users, inflate the token supply, or destroy holder balances. The contracts recommend a multisignature owner in their documentation, but this requirement is not enforced on-chain.

```
function pause() external onlyOwner {
    _pause();
}
...
function blacklistAddress(address account) external onlyOwner {
    blacklist[account] = true;
    emit Blacklisted({account: account});
}
...
function setMintCap(uint256 newCap) external onlyOwner {
    mintCapPerAddress = newCap;
    emit MintCapUpdated({newCap: newCap});
}
...
function mint(address to, uint256 amount) external onlyOwner
whenNotPaused {
    ...
    totalMintedBy[to] += amount;
    _mint(to, amount);
}
...
function complianceBurn(address from, uint256 amount) external
onlyOwner {
```

```
    ...
    _burn(from, amount);
    emit Burn({from: from, amount: amount});
}
```

Recommendation

To address this finding and mitigate centralization risks, it is recommended to evaluate the feasibility of migrating critical configurations and functionality into the contract's codebase itself. This approach would reduce external dependencies and enhance the contract's self-sufficiency. It is essential to carefully weigh the trade-offs between external configuration flexibility and the risks associated with centralization.

Team Update

The team has acknowledged that this is not a security issue and states:

This is by design. Just like Tether, we cannot remove these logic controls — they are essential for regulatory compliance. The risk is mitigated by using a multisig wallet as the owner, and the two-step ownership transfer prevents accidental transfers.

FBCBS - Freeze Blocks Compliance Burn Seizure

Criticality	Minor / Informative
Location	evm-PYGT.sol#L202,246 tron-PYGT.sol#L203,246
Status	Unresolved

Description

`_update` applies the freeze check unconditionally to both parties before any other logic, and unlike the blacklist check immediately following it, the freeze check has no `to == address(0)` carve-out for burns. `complianceBurn` guards only on the blacklist flag and relies on the shared hook for everything else, so when an account carries both flags the burn reverts with `AccountFrozen`.

Freezing an account during an investigation and blacklisting it once the status is formalised is a natural ordering for the sanctions workflow this contract is modelled on, which makes the overlapping-flag state routine rather than exceptional. In that state the seizure function that exists specifically to destroy flagged balances cannot execute, and the `_update` documentation's claim that blacklisted accounts may still burn to the zero address is silently false. The owner can recover by unfreezing first, so no external party gains value, but the ordering dependency is undocumented and surfaces only at the moment a time-sensitive order is being executed.

```
if (frozen[from] || frozen[to]) {
    revert AccountFrozen(frozen[from] ? from : to);
}
if (blacklist[from] && to != address(0)) {
    revert AccountBlacklisted(from);
}
```

Recommendation

Decide whether the freeze flag is intended to override the compliance seizure path and make that decision explicit in the code.

HSSA - High S Signatures Accepted

Criticality	Minor / Informative
Location	router-paymaster.sol#L166,169
Status	Unresolved

Description

Signature recovery in the paymaster uses the native ecrecover precompile, which does not reject signatures with an upper range s value. Every valid approval therefore has a second, different byte encoding that also passes validation. The accompanying comment states that there is no malleability risk, which is inaccurate. Off-chain systems that identify or deduplicate approvals by their signature bytes can be misled by the second encoding.

```
// Native ecrecover (no OZ high-s restriction) because:
// - we control the signer, no malleability risk
// - ensures compatibility with any library/wallet
if (ecrecover(hash, v, r, s) != signer) {
    return ("", _packValidationData(true, validUntil, validAfter));
}
```

Recommendation

The team is advised to restrict accepted signatures to a single canonical form and to correct the comment. This will remove the alternative encoding of each approval.

NAAD - Nominal Amounts Assumed Delivered

Criticality	Minor / Informative
Location	router-split-paymaster.sol#L49,53
Status	Acknowledged

Description

`split` accepts an arbitrary caller-supplied token and issues two independent `transferFrom` calls for the nominal `amount` and `fee` values, with no verification of the balance actually delivered to either destination. For a fee-on-transfer, deflationary, or rebasing token, the recipient and the fee wallet each receive strictly less than the requested figure while the payer's allowance and balance are debited the full pre-fee amount on both legs.

Each individual call is correctly written and the control flow completes without reverting, so nothing in the contract signals the shortfall, yet the documentation states that the recipient receives `amount` tokens. Since the token parameter is unconstrained and the contract presents itself as tolerant of non-standard token behaviour, tokens exhibiting this behaviour fall squarely within the input space the contract invites. Downstream logic that reconciles against the requested figures rather than observed balance deltas will consistently over-credit.

```
function split(SplitParams calldata params) external {
    ITRC20(params.token).safeTransferFrom(msg.sender, params.recipient,
params.amount);

    if (params.fee != 0) {
        ITRC20(params.token).safeTransferFrom(msg.sender,
params.feeWallet, params.fee);
    }
}
```

Recommendation

Establish explicitly which token behaviours the router is intended to support and enforce that decision on-chain. Either constrain accepted tokens to those with standard transfer semantics, or measure the destination's balance before and after each leg and treat the

observed delta as the authoritative delivered amount, surfacing it so that integrators reconcile against what actually moved rather than what was requested.

Team Update

The team has acknowledged that this is not a security issue and states:

There is no need to address this because only our own backend assembles these transactions. We control which tokens are used and know they do not exhibit fee-on-transfer or deflationary behavior.

PBAP - Permit Bypasses Account Permissions

Criticality	Minor / Informative
Location	TRON-PYGT.sol#L25,102
Status	Unresolved

Description

TRON accounts can change their owner key or require several signatures through the network's account permission system, while keeping the same address. The Tron PYGT contract inherits TRC20Permit, which approves allowances from any signature that ecrecover maps to the holder's address. That address is derived from the account's original key, and the permit path does not consult the account's on-chain permissions.

A holder who rotates a compromised key, or moves the account under a multisignature permission, remains exposed. Anyone holding the original key can still sign a permit for the full balance and transfer it with transferFrom, bypassing the protection the holder put in place.

```
contract PYGT is TRC20, TRC20Permit, TRC20Pausable, Ownable2Step {
...
    )
        TRC20("PYGT", "PYGT")
        TRC20Permit("PYGT")
        Ownable(initialOwner)
```

Recommendation

The team is advised to evaluate whether permit approvals should be available on Tron, given that they are not bound by account permissions, and to document the behavior for holders. This will let holders who rely on key rotation or multisignature permissions understand the residual risk.

SSTW - Silent Success Token Withdrawals

Criticality	Minor / Informative
Location	router-paymaster.sol#L311
Status	Acknowledged

Description

`withdrawTokens()` makes a low-level call to each supplied token address and interprets a successful call with empty return data as a successful transfer:

```
(bool success, bytes memory data) = tokens[i].call(
    abi.encodeWithSignature("transfer(address,uint256)", to,
amounts[i])
);
if (!(success && (data.length == 0 || abi.decode(data, (bool))))) {
    revert TokenTransferFailed();
}
```

An ordinary address with no deployed code can return that same success/empty-data combination. The zero-address check does not exclude other no-code addresses. An authorized caller can therefore submit a batch that completes successfully even though one or more entries perform no token transfer.

The impact is false-success reporting and unreliable reconciliation of privileged withdrawals. Both administrator and withdrawer roles already permit choosing arbitrary recipients; this finding does not grant new withdrawal authority to an unprivileged caller. That permission boundary makes its impact narrower than the permissionless payment-splitting case described elsewhere in the report.

Recommendation

Use `SafeERC20.safeTransfer()` with the project's reviewed OpenZeppelin version to handle boolean-returning and no-return tokens and reject the no-code case. If retaining a custom wrapper, verify deployed code before treating an empty successful return as a token transfer.

Team Update

The team has acknowledged that this is not a security issue and states:

The finding is technically correct, but in practice it is irrelevant — only we (our team) call this method, and we only ever pass addresses of existing, known tokens.

SIDC - Sponsorship Ignores Delegate Changes

Criticality	Minor / Informative
Location	router-paymaster.sol#L189,195,200
Status	Acknowledged

Description

For an EIP-7702 operation, the paymaster hashes a constant marker instead of the sender's current delegated implementation. An account owner can change the delegate while retaining the same paymaster approval, then authorize the EntryPoint's changed operation hash. The new implementation may interpret the approved calldata differently, although signed gas limits and nonce rules still apply.

```
bytes memory initCode = userOp.initCode;
if (
    initCode.length == 2 &&
    initCode[0] == 0x77 &&
    initCode[1] == 0x02
) {
    initCode = hex"770200000000000000000000000000000000000000";
}
```

Recommendation

Bind sponsorship to the delegated implementation using semantics consistent with the configured EntryPoint, and apply the same hashing rule in the off-chain signing service.

Team Update

The team has acknowledged that this is not a security issue and states:

In practice, nothing could be done to exploit this because only we (with the `SIGNER_ROLE`) sign operations and assemble transactions in our backend. No third party is involved in the sponsorship flow.

Functions Analysis

Contract	Type	Bases		
	Function Name	**Visibility**	**Mutability**	**Modifiers**
PYGT	Implementation	TRC20, TRC20Permit, TRC20Pausable, Ownable2Step		
		Public	✓	TRC20 TRC20Permit Ownable
	pause	External	✓	onlyOwner
	unpause	External	✓	onlyOwner
	freeze	External	✓	onlyOwner
	unfreeze	External	✓	onlyOwner
	blacklistAddress	External	✓	onlyOwner
	unblacklistAddress	External	✓	onlyOwner
	setMintCap	External	✓	onlyOwner
	mint	External	✓	onlyOwner whenNotPaused
	burn	External	✓	whenNotPaused
	complianceBurn	External	✓	onlyOwner whenNotPaused
	rescueTokens	External	✓	onlyOwner whenNotPaused
	_update	Internal	✓	

SplitPayment	Implementation			
	split	External	✓	-
	_safeTransferFrom	Private	✓	
SimplePaymasterV8	Implementation	IPaymaster, AccessControl		
		Public	✓	-
	grantRole	Public	✓	-
	revokeRole	Public	✓	-
	validatePaymasterUserOp	External		-
	postOp	External		-
	_validatePaymasterUserOp	Internal		
	_postOp	Internal		
	getHash	Public		-
	parsePaymasterAndData	Public		-
	deposit	Public	Payable	-
	withdrawTo	Public	✓	onlyRole
	addStake	External	Payable	onlyRole
	getDeposit	Public		-
	unlockStake	External	✓	onlyRole
	withdrawStake	External	✓	onlyRole
	withdrawEther	External	✓	onlyRole
	withdrawTokens	External	✓	-

	_requireFromEntryPoint	Internal		
		External	Payable	-

Summary

Viruex implements PYGT tokens on EVM and Tron, a router that splits token payments, and an ERC-4337 paymaster that sponsors signer-approved UserOperations. This audit investigates security issues, business logic concerns and potential improvements.

Disclaimer

The information provided in this report does not constitute investment, financial or trading advice and you should not treat any of the document's content as such. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes nor may copies be delivered to any other person other than the Company without Cyberscope's prior written consent. This report is not nor should be considered an "endorsement" or "disapproval" of any particular project or team. This report is not nor should be regarded as an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Cyberscope to perform a security assessment. This document does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors' business, business model or legal compliance. This report should not be used in any way to make decisions around investment or involvement with any particular project. This report represents an extensive assessment process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk Cyberscope's position is that each company and individual are responsible for their own due diligence and continuous security Cyberscope's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies and in no way claims any guarantee of security or functionality of the technology we agree to analyze. The assessment services provided by Cyberscope are subject to dependencies and are under continuing development. You agree that your access and/or use including but not limited to any services reports and materials will be at your sole risk on an as-is where-is and as-available basis Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives false negatives and other unpredictable results. The services may access and depend upon multiple layers of third parties.

About Cyberscope

Cyberscope is a TAC blockchain cybersecurity company that was founded with the vision to make web3.0 a safer place for investors and developers. Since its launch, it has worked with thousands of projects and is estimated to have secured tens of millions of investors' funds.

Cyberscope is one of the leading smart contract audit firms in the crypto space and has built a high-profile network of clients and partners.



A **TAC Security** Company

The Cyberscope team

cyberscope.io